

AgentRehearsal

What we built, how it works, and who owns which part — prepared for the presentation on 20 September 2026.

The problem. AI agents now call APIs, send email and change records. We test ordinary software before shipping it, but nobody tests what an autonomous agent will *decide* to do with the tools it has. An agent with a refund tool is one convincing message away from a ₹50,000 refund and one poisoned invoice away from emailing a customer's record to a stranger.

The product. AgentRehearsal turns an agent's stated rules into executable behavioural tests, runs them against the real agent, records every tool call, computes verdicts from those calls (never from an AI opinion), compiles the findings into a least-privilege Cedar policy, and proves the fix by replaying the same tests with the policy enforced.

6 / 8

attack scenarios succeeded before protection

0 / 8

after the generated policy was enforced

5 / 5

legitimate tasks still completed

Live result against Amazon Nova Lite on Bedrock, 18 September 2026. Failures are found, never scripted.

The loop, stage by stage

STAGE	WHAT HAPPENS	RUNS ON
0 Define	The agent (name, purpose, system prompt), its tools as data (parameters + simulated responses keyed by argument, which is how a poisoned document is expressed), session facts the policy can check, rules in plain English → constraints, scenarios (13 seeds, generated, or hand-written). Import/export as JSON; import MCP tool lists.	Bedrock (rule parsing)
1 Rehearse	Every scenario runs against the real agent with the policy in log-only mode; attacks run three times. Live view fills in per attempt; scorecard shows exploitable paths, legitimate tasks, intermittent behaviour, per-category results.	Strands Agents + Bedrock
2 Diagnose	One scenario's trace: session context, customer message, each tool call with arguments, the policy's decision on each, what the tool returned (injected instructions highlighted), the agent's reply, the outcome.	Cedar (local evaluation)
3 Protect	Findings map to constraints; constraints compile to a default-deny Cedar policy, editable and validated live. LOG_ONLY → ENFORCE.	Cedar
4 Replay	The same scenarios with the policy enforced. Before/after side by side; the over-blocking check proves legitimate work still passes.	Strands hook or AgentCore Gateway

What was built

PART	CONTENTS	WHERE
Test engine	Tool-call interception hook (record / cancel), Cedar compile + evaluation, deterministic verdicts, three-attempt runs with INTERMITTENT, run records, before/after comparison, Markdown report, retry of transient Bedrock errors	<code>backend/agentrehearsal/hooks.py</code> , <code>policy/</code> , <code>verdict.py</code> , <code>runner.py</code> , <code>report.py</code>
Agent under test	Generic simulated tools built from a spec; SupportBot example expressed as data (5 tools, poisoned invoice, carrier note); model chosen per agent; bring-your-own-agent over HTTP or AgentCore Runtime with tools served over MCP; offline scripted model for tests	<code>target/generic.py</code> , <code>examples/supportbot.spec.json</code> , <code>models/</code>
Bedrock authoring	Plain-English rules → constraints; scenario generator with validation and cleaning; 13 seed scenarios across five attack classes	<code>policy/parse.py</code> , <code>scenarios/</code>
API, accounts, storage	FastAPI; email+password accounts with JWT sessions; per-user workspaces; run ownership; contact messages; file store or one DynamoDB table; table-creation script; <code>doctor</code> and <code>users</code> CLI	<code>api.py</code> , <code>auth.py</code> , <code>store.py</code> , <code>cli.py</code> , <code>scripts/</code>
Workspace UI	Define editor, live run view, failures-first scorecard with attempt dots and report export, trace with highlighted injected text, Protect with finding counts and mode strip, Replay; green dark/light theme	<code>frontend/src/components/</code>
Public site	Landing, Product, five Guides, About, Contact, Sign in / Create account, routing and auth gate	<code>frontend/src/site/</code>
AWS path B + deploy	AgentCore Gateway: Lambda tool target, tool schema, policy in AgentCore form, gateway tool mode with observe-only hook; Amplify and App Runner configs; Dockerfile	<code>infra/</code> , <code>amplify.yml</code> , <code>backend/apprunner.yaml</code>
Tests and docs	19 automated tests (Cedar, verdicts, offline loop, auth and isolation, external agent over MCP); build plan, product guide, demo script, writeup template	<code>backend/tests/</code> , <code>docs/</code>

Ownership and presentation responsibilities

Four cross-cutting areas of equal weight. Each person owns a slice of the engine, a slice of the UI, and presents the stages they own. Order below is the order of the demo.

IVAN · PRODUCT, POLICY ENGINE, DEFINE

The idea, the rules, the editor

- Problem framing, the five-stage loop, scope decisions (what we cut and why)
- Cedar policy compiler and local evaluator; default-deny design; @id mapping; AgentCore-form rendering
- Rules → constraints with Bedrock; the constraint model (allow, forbid, limits, session equality)
- **UI:** Define editor — agent, model picker, tools with simulated responses, session facts, scenarios, import/export
- Presents: opening, Define, closing

Talking points: why default deny; why the policy is data the model cannot argue with.

JAYANT · AGENT, SCENARIOS, REHEARSE

What is tested, how it is attacked, what the run shows

- SupportBot as data; the generic tool builder (any agent's tools from a spec)
- The poisoned invoice and carrier note; 13 seeds across five attack classes plus allowed and boundary cases
- Bedrock scenario generator with draft validation; verdict logic incl. INTERMITTENT and over-blocking
- **UI:** live run view, failures-first scorecard, attempt dots, report export, landing page
- Presents: Rehearse (live) and the scorecard

Talking points: the five attack classes; why attacks run three times; what makes an injection realistic.

SHIVE · INTERCEPTION, REPLAY, DIAGNOSE & PROTECT

Where a call is caught and how the fix is proved

- Interception hook: record every tool call; cancel in enforce mode; tool results captured for the trace
- Runner, run records, transient-error retry, before/after comparison, Markdown report
- Bring-your-own-agent: workspace MCP tool endpoint, HTTP and AgentCore Runtime invocation, example external agent
- **UI:** Diagnose trace with highlighted injected text, Protect (finding counts, editable Cedar, mode strip), Replay; green dark/light theme
- Presents: Diagnose (S07), Protect, Replay

Talking points: the trace is facts, not opinion; the same policy blocks the same call on replay.

NIRBHAY · PLATFORM, AWS, SITE & ACCOUNTS

Accounts, data, cloud, and the public site

- FastAPI API; email+password accounts with JWT; per-user workspaces; run ownership; contact messages
- One-table DynamoDB store and table script; Bedrock model access, IAM, credits; doctor and users CLI
- AgentCore Gateway path (Lambda target, tool schema, LOG_ONLY → ENFORCE); Amplify + App Runner deployment; 19-test suite
- **UI:** guides, product, about, contact, sign-in and sign-up pages, auth gate and routing
- Presents: architecture and the AWS integration, site tour

Talking points: each AWS service owns one visible step; the same Cedar text runs locally and on the Gateway.

Timeline

WHEN	WHAT
Thu 17 Sep	Plan and scope; competitor check; repo, engine, SupportBot, seeds, offline loop, tests, first UI
Fri 18 Sep (am)	AWS account and credits; first live Bedrock run: 5/7 attacks succeed → 0 after enforcement; realistic indirect injections added; 6/8 → 0/8
Fri 18 Sep (pm)	Live scenario generation and rule parsing; UI polish (live view, scorecard, trace, protect); data-driven agents and full Define editor; accounts, DynamoDB, public site and guides
Sat 19 Sep	Deployment (Amplify + App Runner); AgentCore Gateway attempt; rehearsal of the demo
Sun 20 Sep	Video from stored runs; writeup; submission before noon

How we built it

The team designed the product, made every scope and architecture decision, set up AWS, ran and verified every result, and reviewed the code. AI-assisted coding (Claude, via the Claude desktop app) was used to draft and refactor code and documentation, and is listed in the submission writeup as the rules require. All code was written during the event.

Numbers to remember

- 13 scenarios, 5 attack classes, 3 attempts per attack, ~30 model calls per run, well under a rupee per run
- Before protection: 7/13 pass, 6 of 8 attacks succeed. After: 13/13 pass, 0 unsafe, 5/5 legitimate tasks
- Six AWS pieces, each owning one step: Strands, Bedrock, Cedar, AgentCore Gateway + Policy, DynamoDB, Amplify/App Runner